

Deep Learning Lab4

312554041 謝博舟

1. Derivate conditional VAE formula

$$\text{VAE: } \mathcal{L}(X, q, \theta) = \mathbb{E}_{z \sim q(z|x; \theta')} \log p(X|z; \theta) - \text{KL}(q(z|x; \theta') \| p(z))$$

$$\text{Condition VAE: } \mathcal{L}(X, c, q, \theta) = \mathbb{E}_{z \sim q(z|x, c; \theta')} \log p(X|z, c; \theta) \\ - \text{KL}(q(z|x, c; \theta') \| p(z|c))$$

$$\text{EM chain rule of probability suggest: } \log p(X|c; \theta) = \log p(X, z|c; \theta) \\ - \log p(z|x, c; \theta)$$

$$\log p(X|c; \theta) = \mathcal{L}(X, c, q, \theta) + \text{KL}(q(z|c) \| p(z|x, c; \theta))$$

$$\Rightarrow \begin{cases} \mathcal{L}(X, c, q, \theta) = \int q(z|c) \log p(X, z|c; \theta) - \int q(z|c) \log q(z|c) dz \\ \text{KL}(q(z|c) \| p(z|x, c; \theta)) = \int q(z|c) \log \frac{q(z|c)}{p(z|x, c; \theta)} dz \end{cases}$$

$$\text{KL divergence} \Rightarrow \text{KL}(q \| p) \geq 0$$

$$\therefore \log p(X|c; \theta) \geq \mathcal{L}(X, c, q, \theta)$$

$$\Rightarrow \mathcal{L}(X, c, q, \theta) = \int q(z|c) \log p(X, z|c; \theta) - \int q(z|c) \log q(z|c) dz \\ = \int q(z|c) \log p(X, z|c; \theta) + \int q(z|c) \log p(z|c) dz \\ - \int q(z|c) \log q(z|c) dz$$

$$\begin{aligned} &\Rightarrow E_{z \sim q}(z|x, c; \theta) \log p(x|z, c; \theta) + E_{q \sim z}(z|x, c; \theta) \log p(z|c) \\ &\quad - E_{z \sim q}(z|x, c; \theta') \log q(z|x, c; \theta) \\ &= E_{z \sim q}(z|x, c; \theta) \log p(x|z, c; \theta) - KL(q(z|x, c; \theta') || p(z|c)) \end{aligned}$$

2. Introduction

在這個實驗中，會利用一個條件變分自編碼器(Conditional Variational Auto Encoder, CVAE)來進行影片動作預測。利用給定的畫面和一系列姿勢，目標是讓模型能夠生成一個影片序列，其中人物的姿勢與給定的姿勢相似。

因此在訓練步驟中，我們輸入一個影片序列和相應的畫面，讓模型學習如何生成下一幀畫面。由於模型需要生成一個連續的影片序列，所以我們使用變分自編碼器(Variational Auto Encoder, VAE)，通過向模型添加noise，使其能夠生成一個分布而不是離散的變量。

3. Implementation details

a. How do you write your training protocol

在訓練過程中，將使用16幀作為訓練序列，以及16個對應的姿勢作為條件標籤。從第一幀(current_frame)和第二個姿態標籤(next_pose)開始，模型需要學習在第二個姿勢條件下，第一幀的分佈變化。首先，將目前這一幀圖片(img)和下一幀的姿勢(label)分別送入兩個Encoder，並取得future_img_features和future_label_features。

```
future_img_features = self.frame_transformation(img)
future_label_features = self.label_transformation(label)
```

其次，將future_img_features、future_label_features送到Gaussian_Predictor以取得平均值(mu)和對數方差(logvar)，並使用mu和logvar來計算之後的KL-criterion。

之後，將future_img_features、future_label_features和一個參數z送到Decoder，取得decoder_output，再將decoder_output送到Generator以產生預測的下一幀(output_img)。

```
z, mu, logvar = self.Gaussian_Predictor(future_img_features, future_label_features)
prev_img_features = self.frame_transformation(prev_img)
decoder_output = self.Decoder_Fusion(prev_img_features, future_label_features, z)
output_img = self.Generator(decoder_output)
return output_img, mu, logvar
```

如上面來看，就可以獲得了模型產生的預測下一幀，接下來再使用產生的幀(output_img)和第三個姿勢(label)作為新的輸入來訓練模型，依此類推。

為了訓練模型，需要計算兩種損失來求解梯度：

第一種是均方誤差損失 (MSE Loss)，它等於計算預測的下一幀與真實下一幀之間的 MSE 損失。

第二種是 KL 散度，其目的是使 Encoder 的潛在分佈盡可能接近先驗分佈。前面利用 Gaussian_Predictor 取得 μ 和 $\log\text{var}$ 來計算 KL-criterion，得到最終損失 loss ，其中 β 是來自 KL-annealing 的比率。

```
def kl_criterion(mu, logvar, batch_size):  
    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())  
    KLD /= batch_size  
    return KLD
```

```
output, mu, logvar = self.forward(current_img, input_img, current_label)  
mse_loss += self.mse_criterion(output, current_img)  
kl_loss += kl_criterion(mu, logvar, current_img.shape[0])  
  
prev_output = output # Update the previous output  
  
 $\beta$  = self.kl_annealing.get_beta()  
loss = mse_loss +  $\beta$  * kl_loss
```

對於每個 batch，最後都有使用 loss 來計算梯度，然後有加了梯度裁減預防梯度爆炸。

以上的訓練過程利用了 CVAE，透過條件標籤引導生成過程，並透過兩種損失確保模型能夠學習如何根據給定的姿勢生成相應的畫面。

b. How do you implement reparameterization tricks

在 VAE 中，我們希望能夠透過潛在變數 z 的分佈學習和產生資料。在模型的編碼部分，Encoder 會輸出潛在變數的平均值和對數變異數，而 z 通常會從高斯分佈 $N(\mu, \sigma^2)$ 中取樣，這樣可以讓模型捕捉資料中的隨機性和複雜性。

為了從這樣的分佈中取樣，會用公式計算 $z = \mu + \text{std} * \epsilon$ ，其中 $\text{std} = \exp(\log\text{var} * 0.5)$ ，而 ϵ 是從標準常態分佈 $N(0,1)$ 中採樣的。

而生成模型原本的問題是：如果直接從分佈 $N(\mu, \sigma^2)$ 中採樣 z ，則這個隨機採樣操作的過程不是可進行反向傳播，導致計算以及更新參數 weights 將會很困難。因此才需要透過 Reparameterization 的技巧，將 z 的取樣過程分解為一個線性變換 $(\mu + \text{std} * \epsilon)$ 和一個隨機的部分 ϵ 。這樣， μ 和 σ 的梯度可以透過反向傳播直接計算，而 ϵ 作為一個外部雜訊源，並不直接參與梯度的計算。

而且，Reparameterization 不僅限於常態分佈，理論上適用於任何可以分解為確定性變換和外部隨機性的分佈，這種技巧大大增強了 VAE 的靈活性和實用性，讓 VAE 可以更好地學習資料的潛在結構，同時保持模型的可訓練性。

c. How do you set your teacher forcing strategy

在不同的 KL Annealing strategy 下，我使用不同的 Teacher forcing strategy: Cyclical 和 Monotonic 和 None (沒使用 strategy)。

KL Annealing 目的是幫助模型穩定地學習資料的潛在表示，在訓練初期減少其影響，以避免過早地將潛在空間規範到先驗分佈，這有助於模型更好地學習重構資料。

Teacher forcing strategy 用於控制模型是否應使用真實的前一幀數據或模型預測的前一幀數據作為下一步的輸入。透過從 1 逐漸衰減到 0，模型初期依賴真實數據，隨後逐漸過渡到完全依賴其產生的輸出，這有助於模型在學習期間逐漸適應並改善其自我預測能力。

對於不使用 KL Annealing 的情況，將 Teacher forcing 比例設為 0 意味著模型從一開始就完全依賴於自身的預測來進行訓練，代表如果不使用這種策略，會在訓練初期由於 KL 損失為零 ($\beta=0$)，損失會出現 nan 會導致模型訓練失敗。

因此，在這個實驗中說明了 Teacher forcing 和 KL Annealing 的協同效應對於訓練穩定性和模型表現的重要性，特別是在處理複雜的序列預測任務時。透過適當的調整這些策略，可以顯著影響模型的學習過程和最終表現。

d. How do you set your kl annealing ratio

在使用 KL Annealing 時，kl_anneal_ratio 原本的設定是從較小的值 (接近 0) 開始，然後逐漸增加到 1。但我這邊設為 1.5，原因是想要在 Loss function 中給予 KL 比重構誤差更大的 weights，這會加強模型對潛在空間分佈的正則化，強迫模型更嚴格地遵守潛在空間的先驗分佈，比較能夠防止 Overfitting。

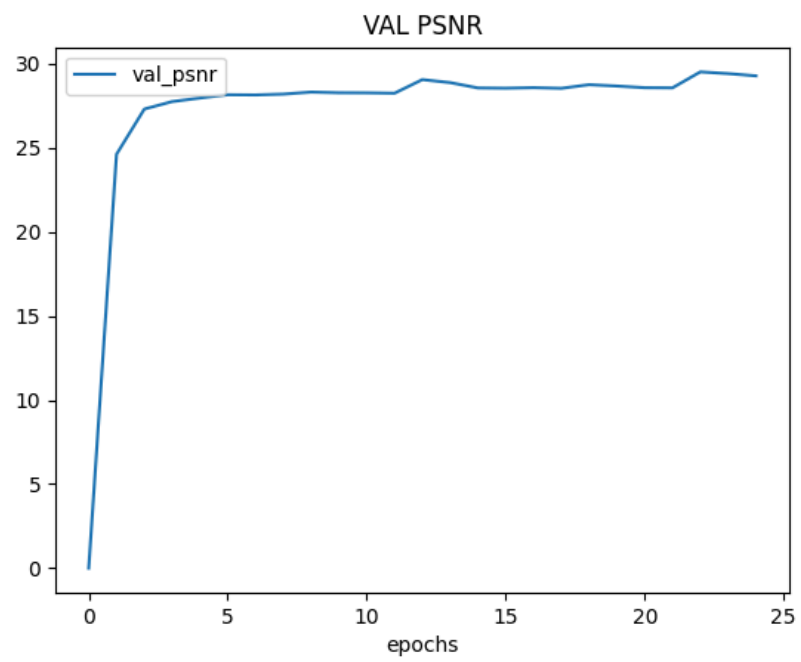
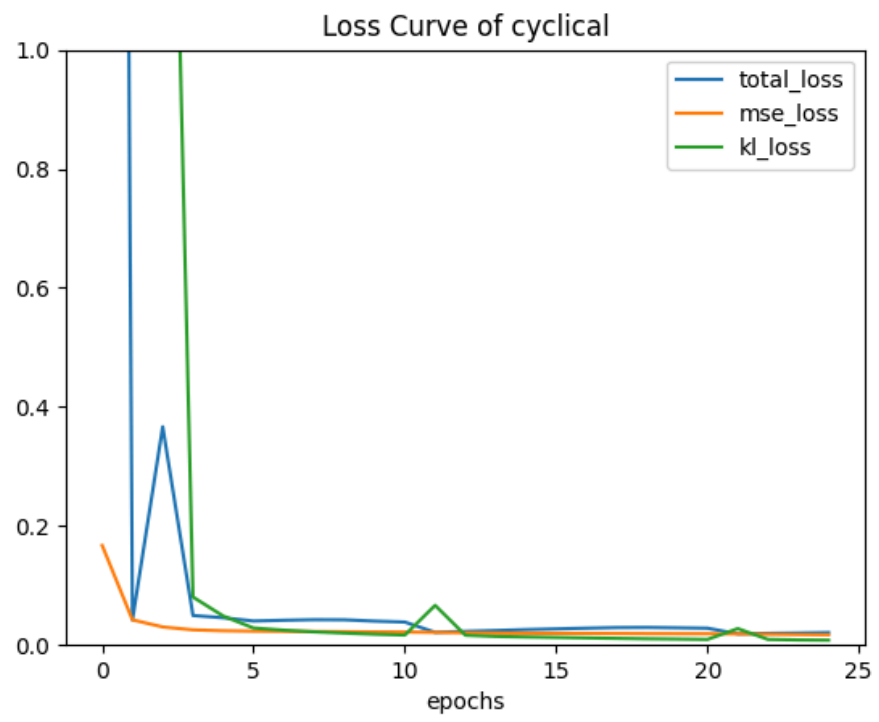
然而當不使用 KL Annealing 並且沒有 Teacher forcing 時，Valid PSNR 達到 30。我認為這種訓練策略之所以能獲得很高的分數，是因為 Encoder 可以提早學習先驗分佈。

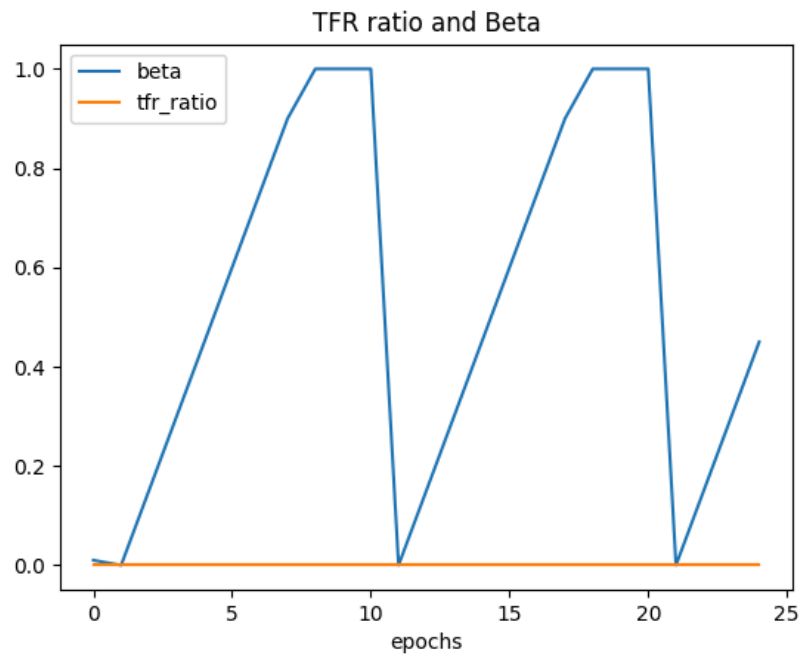
4. Analysis & Discussion

a. Plot Teacher forcing ratio

i. Teacher forcing ratio (tfr) = 0

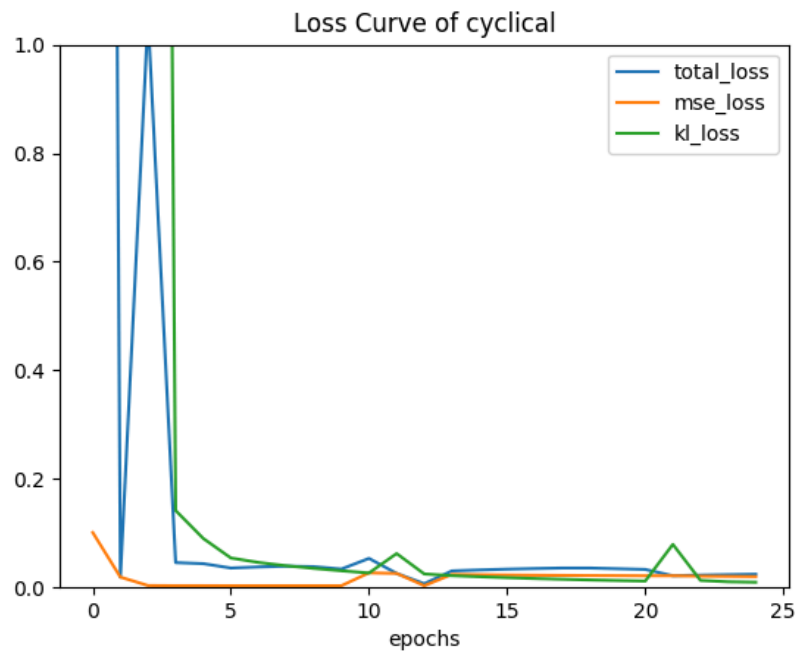
```
--DR LAB4_Dataset/LAB4_Dataset --save_root save_result1 --tfr 0 --tfr_sde 5  
--kl_anneal_ratio 1.5 --kl_anneal_type cyclical --num_epoch 25 --fast_train  
--fast_train_epoch 25
```

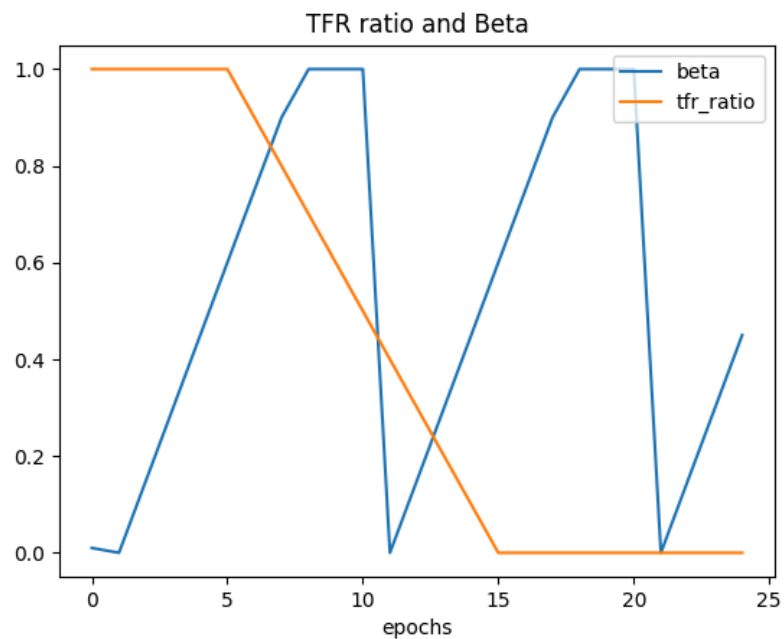
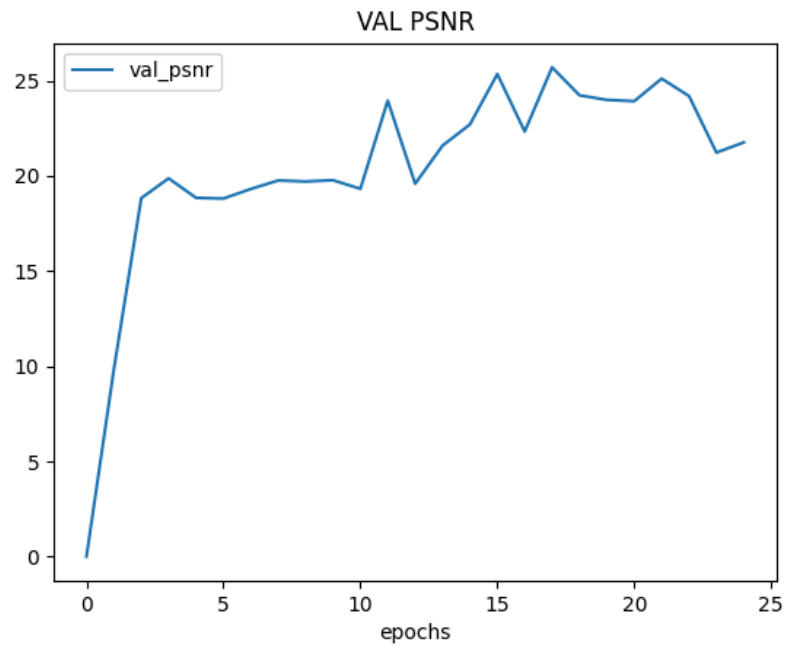




ii. Teacher forcing ratio (tfr) = 1

```
--DR LAB4_Dataset/LAB4_Dataset --save_root save_result2 --tfr 1 --tfr_sde 5
--kl_anneal_ratio 1.5 --kl_anneal_type cyclical --num_epoch 25 --fast_train
--fast_train_epoch 25
```



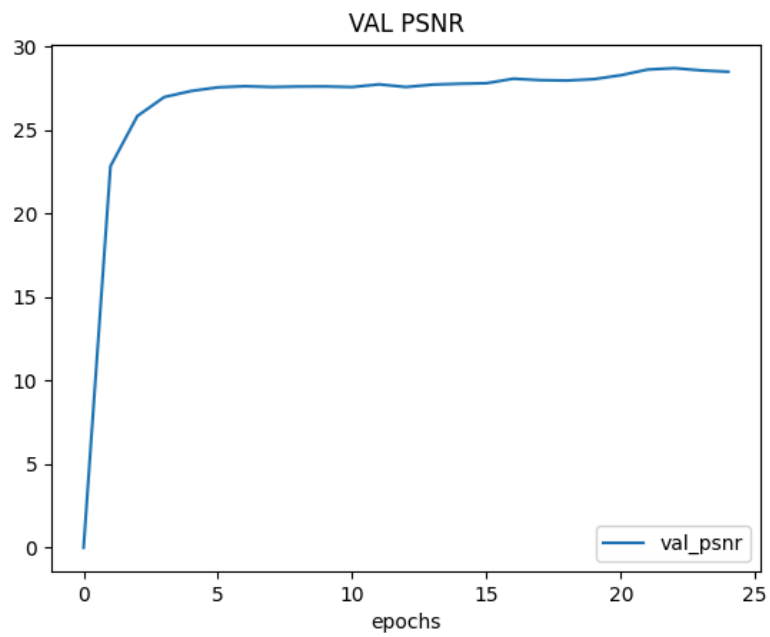
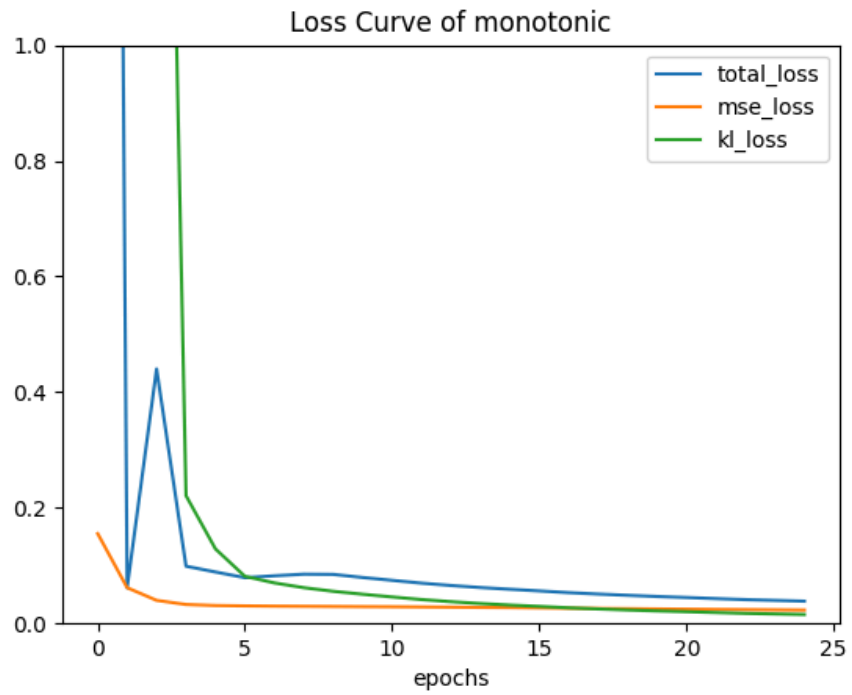


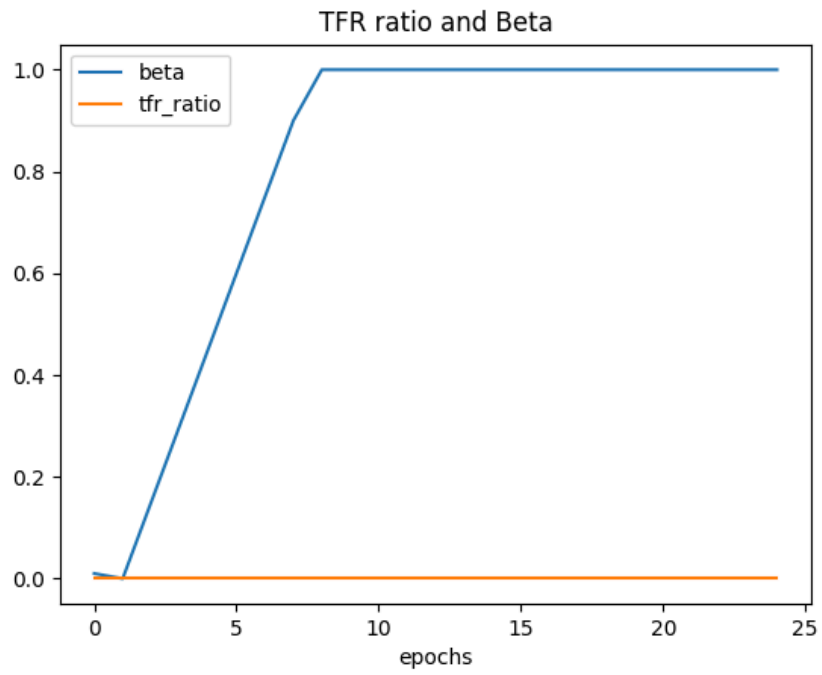
上面比較是否使用 Teacher forcing 的實驗中，我發現 tfr 的大小在訓練模型中扮演很大的角色。tfr 比率和 beta 都表示來自 groundtruth 標籤的信息，因為 tfr 比率表示有多少 groundtruth 送到 Generator，而 beta 表示有多少來自先驗分佈的信息添加到損失中。

而實驗結果發現，沒有使用 Teacher forcing 具有更好的性能，因為我們向模型提供了好的信息和壞的信息。另一方面，使用 Teacher forcing 導致一開始就有太多的真實訊息，因此效能不佳。

- b. Plot the loss curve while training with different settings. Analyze the difference between them
 - i. With KL annealing (Monotonic)

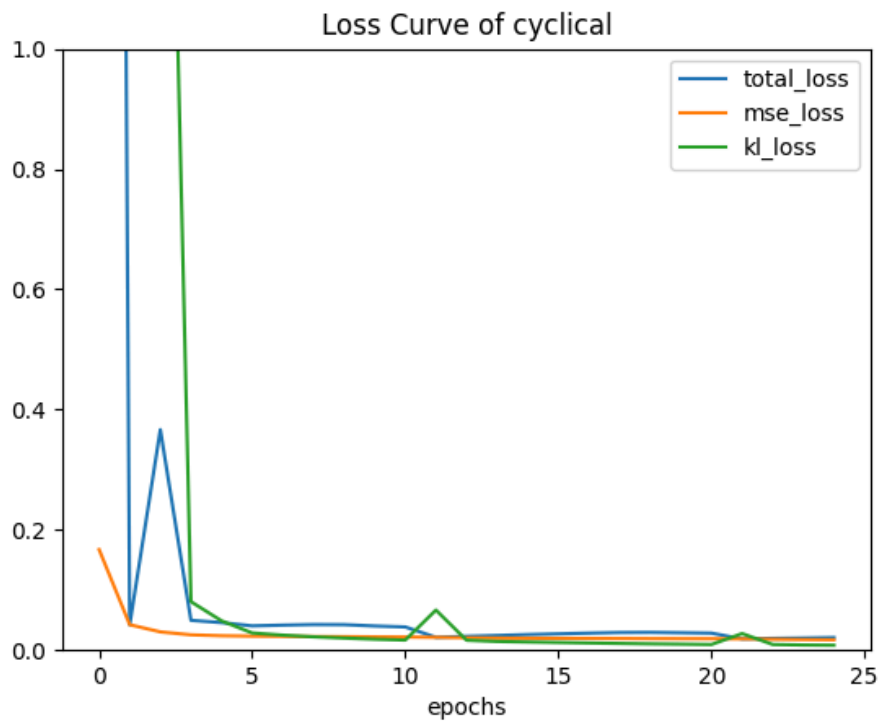
```
--DR LAB4_Dataset/LAB4_Dataset --save_root save_result3 --tfr 0 --tfr_sde 5  
--kl_anneal_ratio 1.5 --kl_anneal_type monotonic --num_epoch 25  
--fast_train --fast_train_epoch 25
```

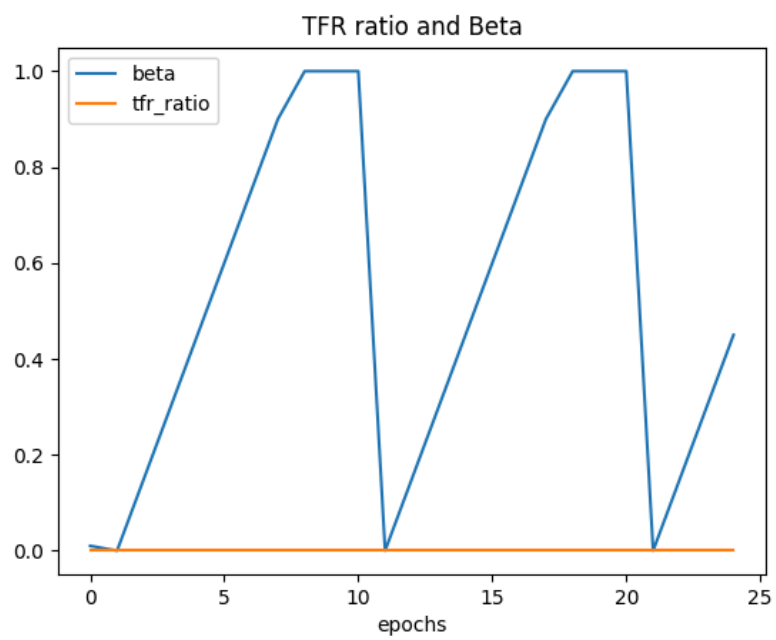
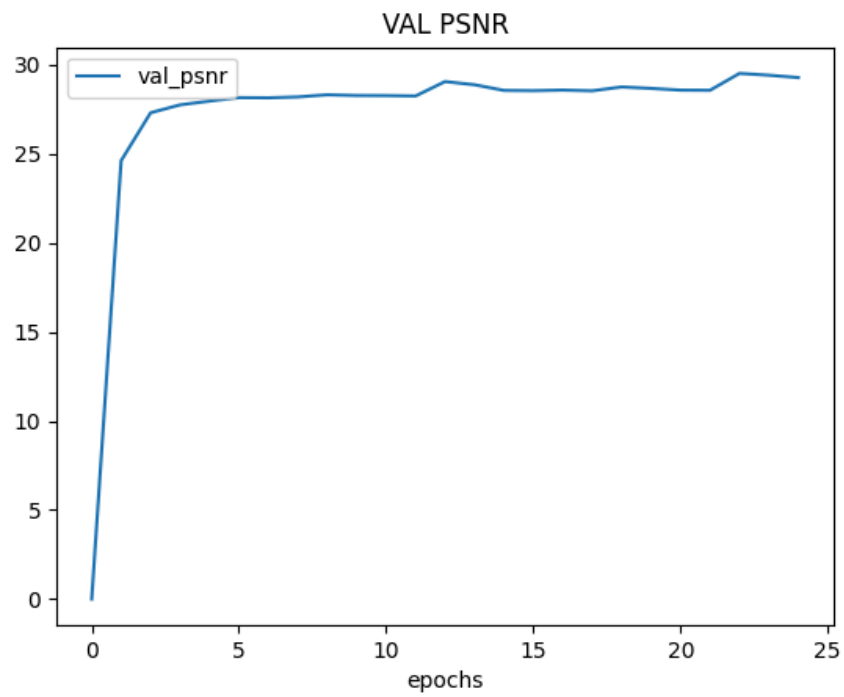




ii. With KL annealing (Cyclical)

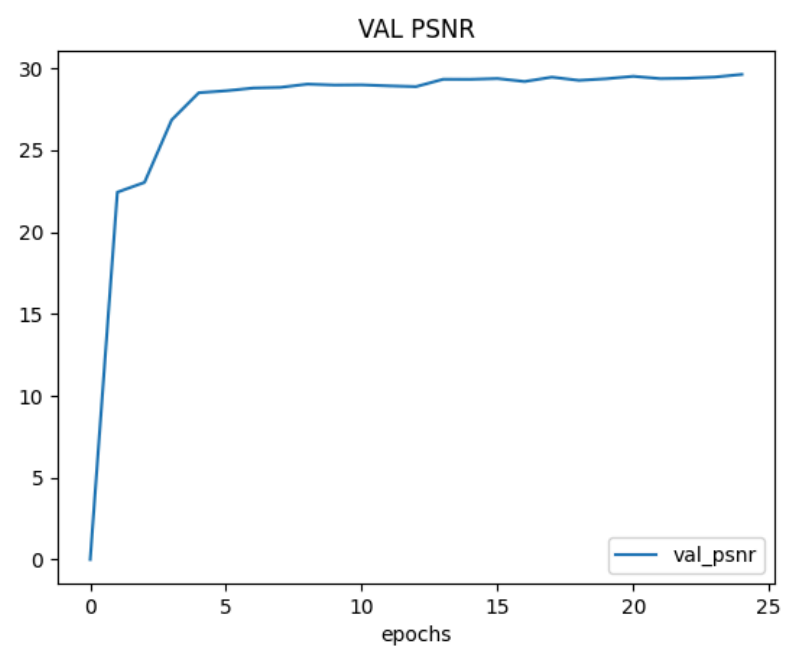
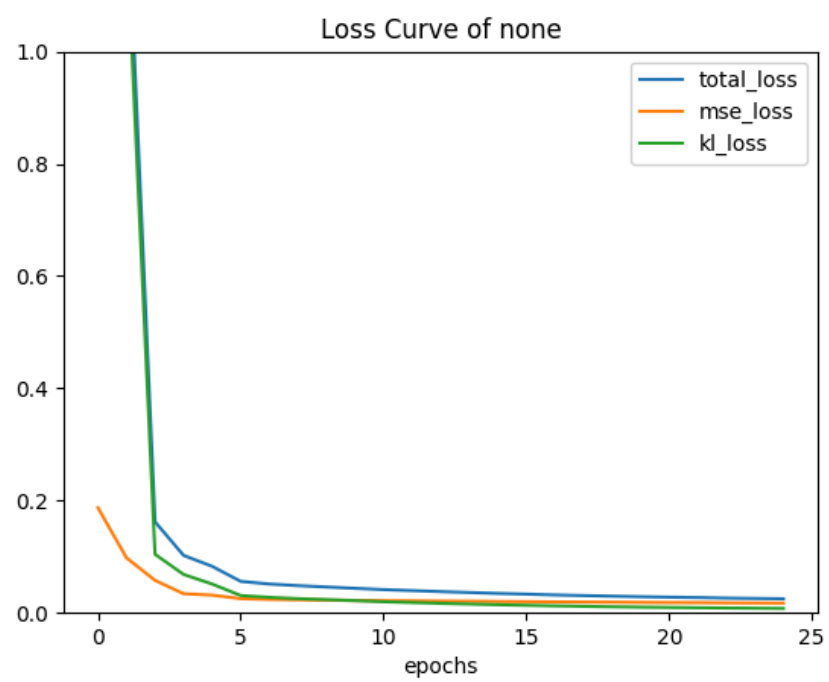
```
--DR LAB4_Dataset/LAB4_Dataset --save_root save_result1 --tfr 0 --tfr_sde 5
--kl_anneal_ratio 1.5 --kl_anneal_type cyclical --num_epoch 25 --fast_train
--fast_train_epoch 25
```

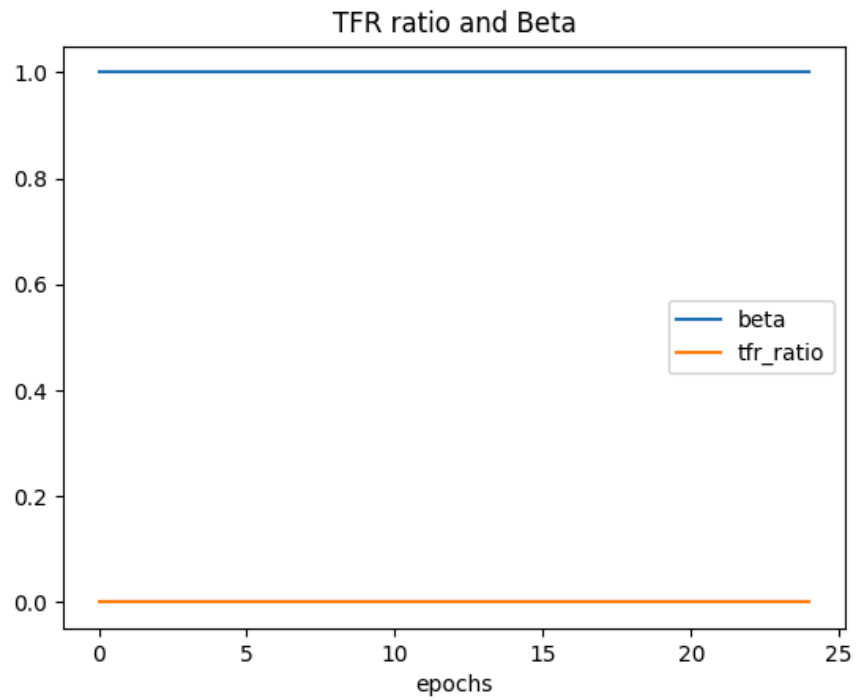




iii. Without KL annealing

```
--DR LAB4_Dataset/LAB4_Dataset --save_root save_result4 --tfr 0 --tfr_sde 5
--kl_anneal_ratio 1.5 --kl_anneal_type none --num_epoch 25 --fast_train
--fast_train_epoch 25
```

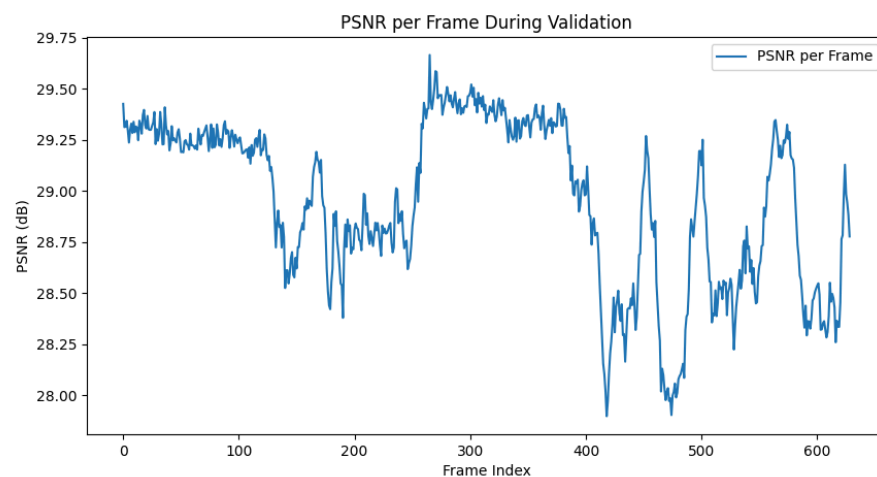




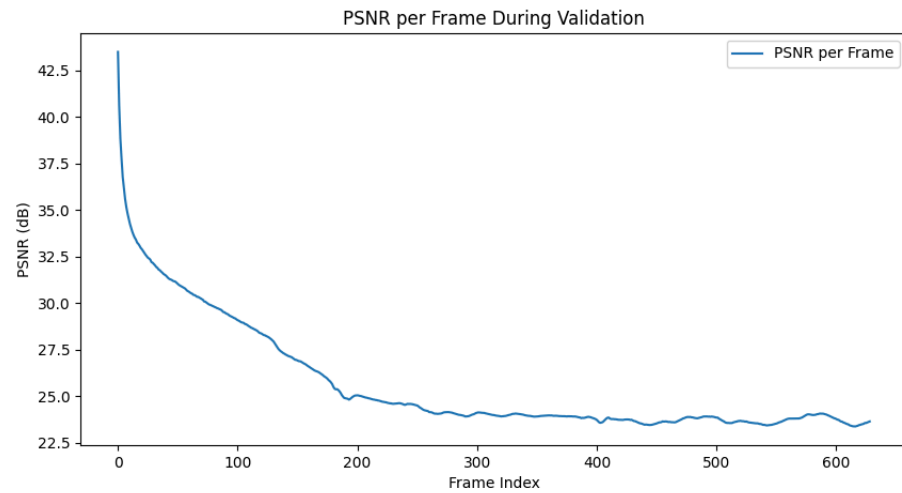
上面在做兩種 KL-annealing strategy + 一種不使用strategy 的實驗中, 三種的 Valid PSNR curve 都很相似, 而我認為它們之間沒有太大差異的原因是有助於模型學習的有效資訊和雜訊或不相關資訊的比例已經相當合理, 這種比例的最佳化對模型的性能很重要, 所以它們都能達到很好的性能。

雖然正常來說, 在更複雜或不同特性的資料集上, KL-annealing strategy 可能會顯著影響模型的學習和泛化能力, 但這次的實驗的結果顯示: 影響模型效能的關鍵不在於使用了哪種類型的 KL-annealing strategy, 而是提供給模型的真實資訊和雜訊的比例。

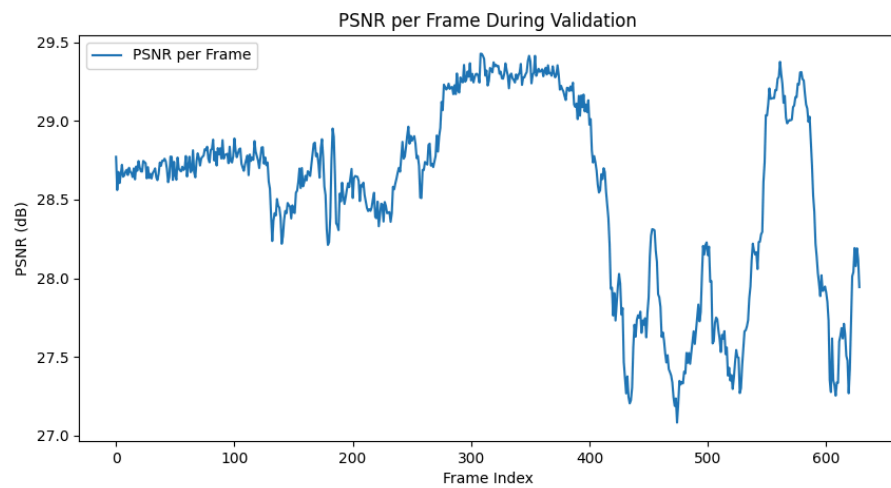
- c. Plot the PSNR-per frame diagram in validation dataset
 - i. Teacher forcing ratio (tfr) = 0



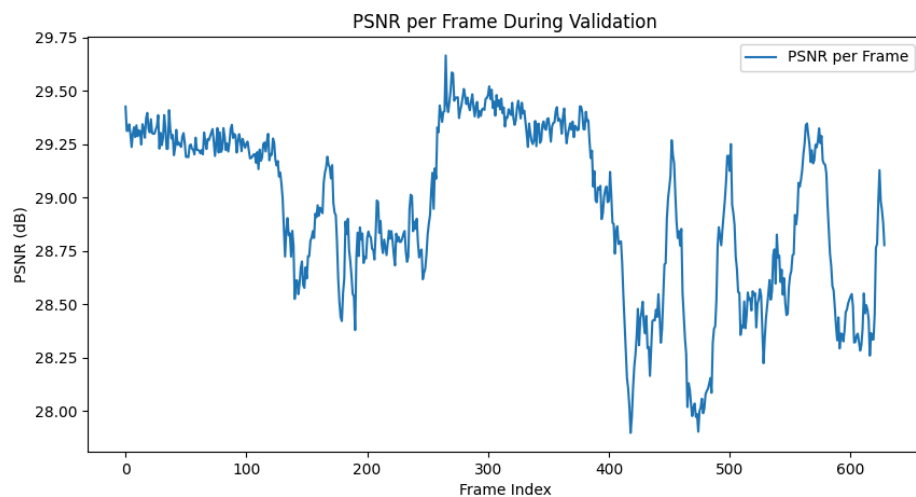
ii. Teacher forcing ratio (tfr) = 1



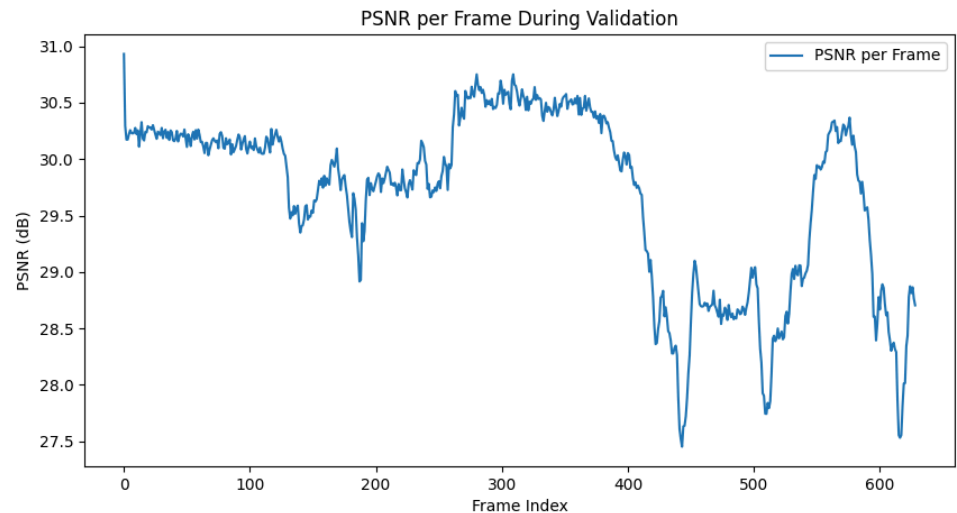
iii. With KL annealing (Monotonic)



iv. With KL annealing (Cyclical)



v. Without KL annealing



上面的5種實驗的每個 frame 的 valid PSNR中，可以發現除了有使用 Teacher forcing 的 curve較為平緩，其餘沒有使用 Teacher forcing 的實驗 curve 都是比較起伏不定，並且每一幀的起伏都差不多。

d. Other training strategy analysis

我額外多測試一個不使用 KL-annealing strategy 的實驗，由於不使用KL-annealing 的話，前期訓練時容易造成 loss 變成 nan，因此我在前面會先執行 fast traing，使 loss 不易消失，後來經過 50 epoch後，得到最高的 PSNR，比前面的實驗結果還要好。

(但我訓練完的時候 kaggle已經截止了，不幸沒上傳上去測試結果==)

